Mathematical Modeling of Malware Spread Using the SIQRAS Model

Problem Statement:

- *Computer viruses and malware spread rapidly through networks, causing massive financial and data losses.*
- *Traditional cybersecurity tools react after infection.*
- *This project uses mathematical modeling, inspired by epidemiology, to predict, analyze, and control malware outbreaks.*

Objective of the Project:

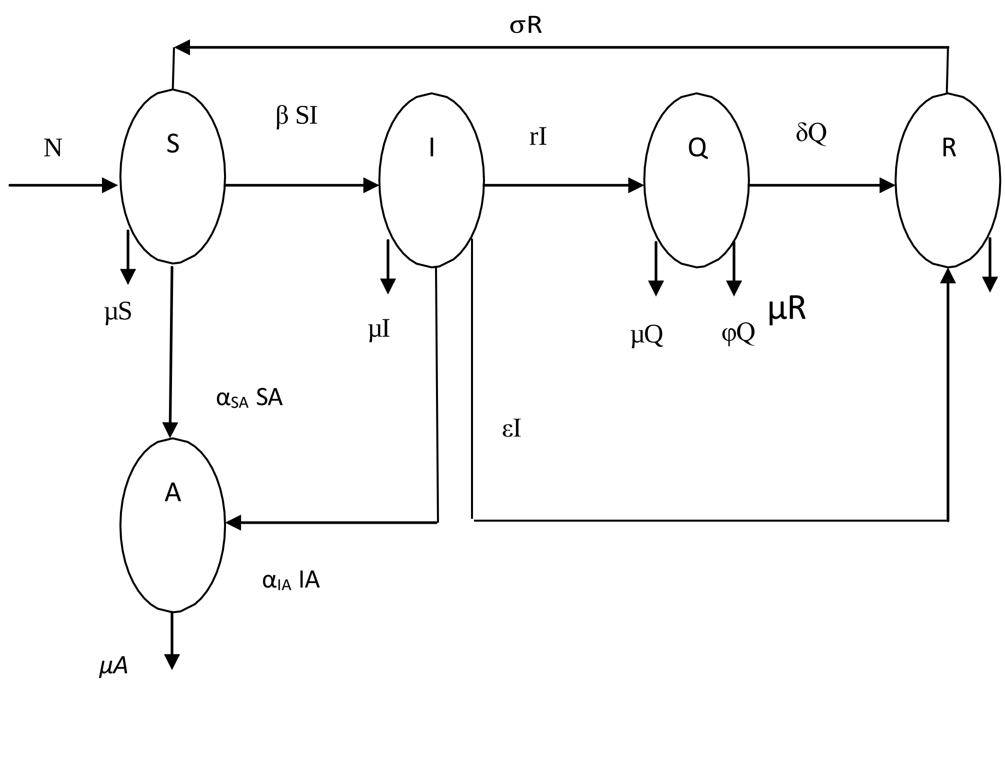
- *To model malware propagation mathematically*
- *To study the effect of antivirus (patching) and quarantine (isolation)*
- *To validate the model using the WannaCry ransomware attack*
- *To visualize results using interactive simulations*

SIQRAS Compartment Model:

The total number of computers in a network is divided into five compartments.

SYMBOL	MEANING	CYBER INTERPRETATION
S	*Susceptible*	*Unpatched, vulnerable systems*
I	*Infected*	*Malware-infected systems*
Q	*Quarantined*	*Isolated infected machines*
A	*Recovered*	*Cleaned systems*
R	*Antidotal*	*Systems with antivirus/patch*
S	*Susceptible*	*Unpatched, vulnerable systems*

SIQRAS MODEL FOR THE TRANSMISSION OF MALICIOUS OBJECTS



Where N be the influx rate that is new nodes to the network and each node is assumed to be susceptible, β is

the per capita contact rate, which is the average number of effective contact with other nodes per unit time, μ denotes death rate due to other than the malicious objects. γ be the rate of quarantined nodes from the infectious class, δ and ϵ be rate of recovery from the quarantined and infectious class respectively. α_{SA} and α_{IA} are rate of conversion from susceptible to antidotal and rate of conversion from infection to antidotal respectively. σ be the rate of transmission of nodes from recovered class to susceptible class. $\emptyset$ be the death rate due to malicious object in Quarantine compartment.
Here the influx rate is considered to be N = 0, representing that there is no incorporation of new nodes in the network during the propagation of the considered viruses, because its action is faster than the network expansion. The same reason justifies the choice of $\mu = 0$, considering that the machine obsolescence time is larger than the time of the virus action.

Variables used in the SIQRAS MODEL:

Variables	**Meaning**
β	*Infection rate due to contact*
γ	*Quarantine rate*
ε	*Recovery rate*
δ	*Recovery from quarantine*
$\alpha_{\square a}$	*Preventive patching*
α_{ia}	*Antivirus cleaning*
σ	*Loss of immunity*
φ	*Failure in quarantine*
μ	*Device removal due to things other than the malicious objects*
N	*Influx rate i.e new nodes are added to the network and each node is assumed to be susceptible*

Assumptions:

- *Network size is constant(N=0)*
- *Malware spreads via contact*
- *Antivirus works perfectly*
- *Quarantine isolates systems completely*
- *No new systems join during outbreak(T=constant)*

- *All parameters are positive*

Differential Equations:

$$\frac{dS}{dt} = N - \beta SI - a_{SA}SA - \mu S + \sigma R$$
$$\frac{dI}{dt} = \beta SI - \mu I - \alpha_{IA}IA - (\varepsilon + \gamma)I$$
$$\frac{dQ}{dt} = \gamma I - (\mu + \phi + \delta)Q$$
$$\frac{dR}{dt} = \delta Q - (\mu + \sigma)R + \varepsilon I$$
$$\frac{dA}{dt} = \alpha_{IA}IA - \mu A + \alpha_{SA}SA$$

Assuming N=0, $\mu = 0$:

$$\frac{dS}{dt} = -\beta SI - a_{SA}SA + \sigma R \qquad .....1$$
$$\frac{dI}{dt} = \beta SI - \alpha_{IA}IA - (\varepsilon + \gamma)I \qquad .....2$$
$$\frac{dQ}{dt} = \gamma I - (\phi + \delta)Q \qquad .....3$$
$$\frac{dR}{dt} = \delta Q - \sigma R + \varepsilon I \qquad .....4$$
$$\frac{dA}{dt} = \alpha_{IA}IA + \alpha_{SA}SA \qquad ....5$$

Conservation of Total Systems(T=constant) :

Mathematically:

- $S, I, Q, R, A \geq 0$
- *Total population constant:*

$$\frac{d}{dt}(S + I + Q + R + A) = 0$$

Adding all equations:

$$\frac{d}{dt}(S + I + Q + R + A)$$

Substituting RHS from each equation:

$$= (-\beta SI - \alpha_{SA}S + \sigma R) + (\beta SI - \alpha_{IA}I - (\varepsilon + \gamma)I)$$
$$+(\gamma I - (\delta + \phi)Q) + (\delta Q - \sigma R + \varepsilon I) + (\alpha_{IA}I + \alpha_{SA}S)$$

Final Result:

$$\frac{dT}{dt} = 0$$

Positivity of Solutions:

If $S(0), I(0), Q(0), R(0), A(0) \geq 0$, then all variables remain non-negative for all $t > 0$.

- *All outflows are proportional to the variable itself*
- *No equation can produce negative values*

Equilibrium Point

Malware-Free Equilibrium:

Setting derivatives to zero:

$$\frac{dS}{dt} = \frac{dI}{dt} = \frac{dQ}{dt} = \frac{dR}{dt} = \frac{dA}{dt} = 0$$

From eq.2:

$$\frac{dI}{dt} = 0 \Rightarrow I = 0$$

From eq.3 and 4:

$$Q = 0, R = 0$$

Using conservation from eq.5:

$$A = T$$

***Final Equilibrium**:*

$$P_1 = (0,0,0,0,T)$$

Stability Analysis:

Malware-Free Equilibrium:

$$P_1 = (0,0,0,0,T)$$

- All computers are protected
- Jacobian eigenvalues are **negative**(verified computationally).
- System is **asymptotically stable**

Malware dies out permanently.

Basic Reproduction Insight :

Infection grows when infection rate dominates removal and quarantine.
Mathematically:

$$\beta S > (\varepsilon + \gamma + \alpha_{IA})$$

This is a **threshold condition.**

Linearization

Stability is analyzed by linearizing the system near equilibrium using the Jacobian matrix.

$$J = \begin{bmatrix} -(\beta I + \alpha_{SA} A) & -\beta S & 0 & \sigma & -\alpha_{SA} S \\ \beta I & \beta I - \alpha_{IA} A - (\epsilon + \gamma) & 0 & 0 & -\alpha_{IA} I \\ 0 & \gamma & -(\phi + \delta) & 0 & 0 \\ 0 & \epsilon & \delta & -\sigma & 0 \\ \alpha_{SA} A & \alpha_{IA} A & 0 & 0 & \alpha_{IA} I + \alpha_{SA} S \end{bmatrix}$$

If P_1 is put into the Jacobian matrix, the eigenvalues can be found.
All eigenvalues have **negative real parts** (verified computationally).

Numerical Method Derivation

Euler's Method Derivation:

Definition:

$$\frac{dX}{dt} \approx \frac{X_{n+1} - X_n}{\Delta t}$$

$$X_{n+1} = X_n + \Delta t \cdot \frac{dX}{dt}$$

Appling to each equation:
Example:

$$S_{n+1} = S_n + \Delta t(-\beta S_n I_n - \alpha_{SA} S_n + \sigma R_n)$$

CORE of the PYTHON CODE :

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.widgets import Slider, Button
```

Explanation:

Libraries Used

- ***NumPy*** *– numerical calculations and arrays*
- ***Matplotlib*** *– plotting graphs*
- ***Matplotlib Widgets*** *– interactive sliders and buttons*

These allow ***real-time visualization*** *of malware spread.*

Code:

```
dt = 0.1
T = 50
time = np.arange(0, T, dt)
```

Explanation:

Time Settings

- *dt = 0.1 → time step*
- *T = 50 → total simulation time*
- *Simulation runs from* ***t = 0 to t = 50***

Smaller dt values improve accuracy.

Code:

```
S0, I0, Q0, R0, A0 = 90, 5, 0, 0, 5
```

Explanation:

Initial Conditions:

Variable	Initial Value	Meaning
S_0	90	Vulnerable systems
I_0	5	Initially infected
Q_0	0	Isolated systems
R_0	0	Recovered
A_0	5	Patched systems

*Total systems = **100***

Model Parameters:

Controllable Parameters (Sliders)

Code:

```
beta0 = 0.01
gamma0 = 0.05
alphaSA0 = 0.01
alphaIA0 = 0.02
```

Explanation:

Default values are given to the **Controllable Parameters (Sliders)**

Controllable Parameters (Sliders)	Interpretation
β	*Malware infection rate*
γ	*Quarantine rate*
αSA	*Patching of susceptible systems*
αIA	*Antivirus cleaning of infected systems*

Fixed Parameters

Code:

```
delta = 0.1
epsilon = 0.03
sigma = 0.01
phi = 0.02
```

Explanation:

Default values are given to the **Fixed Parameters.**

Fixed Parameter	*Meaning*
δ	*Recovery from quarantine*
ε	*Recovery from infection*
σ	*Loss of immunity*
φ	*Quarantine failure*

Differential Equation Logic:

Susceptible (S)

Code:

```
dS = - beta*S[t-1]*I[t-1] - alphaSA*S[t-1]*A[t-1] + sigma*R[t-1]
```

Explanation:

- *Decreases due to infection and patching*
- *Increases when recovered systems lose immunity*

Infected (I)

Code:

```
dI = beta*S[t-1]*I[t-1] - alphaIA*I[t-1]*A[t-1] - (epsilon + gamma)*I[t-1]
```

Explanation:

- *Increases via contact with susceptible systems*
- *Decreases due to quarantine, antivirus, and recovery*

Quarantined (Q)

Code:

```
dQ = gamma*I[t-1] - (phi + delta)*Q[t-1]
```

Explanation:

- *Increases when infected systems are isolated*
- *Decreases via recovery or isolation failure*

Recovered (R)

Code:

```
dR = delta*Q[t-1] - sigma*R[t-1] + epsilon*I[t-1]
```

Explanation:

- *Gained after infection or quarantine*
- *May lose immunity over time*

Antidotal (A)

Code:

```
dA = alphaIA*I[t-1]*A[t-1] + alphaSA*S[t-1]*A[t-1]
```

Explanation:

Grows when:

- *Infected systems are cleaned*
- *Susceptible systems get patched*

Numerical Method:

Code:

```
S[t] = S[t-1] + dS*dt
```

Explanation:

The model uses the ***Euler Forward Method****:*

$$X(t + \Delta t) = X(t) + \frac{dX}{dt} \cdot \Delta t$$

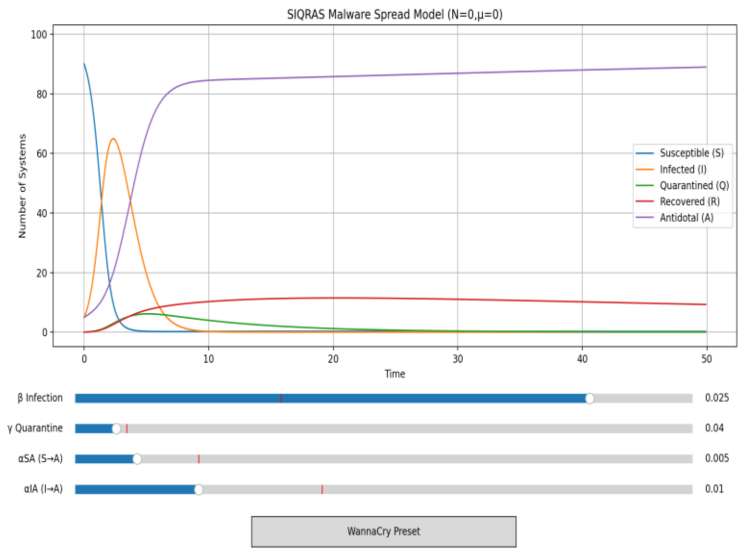
This updates each compartment step-by-step.

---

1. **Interactive Features**

   Link for interactive graph:

- https://rishu-shu.github.io/SIQRAS_model_graph/
- **Sliders** dynamically change parameters
- **Real-time graph updates**
- **WannaCry Preset Button** simulates:
  - High infection rate
  - Delayed quarantine
  - Weak patch deployment



Code:

```
def wannacry(event):
    s_beta.set_val(0.025)       # very high infection
    s_gamma.set_val(0.04)       # delayed quarantine
    s_alphaSA.set_val(0.005)    # weak early patching
    s_alphaIA.set_val(0.01)     # slow antivirus cleanup
```

Real-World Application

This model helps understand:

- Malware outbreak dynamics
- Effectiveness of quarantine & patching
- Importance of early antivirus response